Roadmap IC Design 12 สัปดาห์ — ฉบับแบ่งรายวัน

จากศูนย์สู่ดีไซน์ชิปตัวแรก · วันละ 1–2 ชั่วโมง · 5 วันเรียน + 1 วันทบทวน + 1 วันพัก ต่อสัปดาห์ (รวม 84 วัน)

วิธีใช้

1. ทำตามลำดับ 📖 เรียน (20–30 นาที) → 🛠️ ลงมือ (40–60 นาที) → ✅ Checkpoint ถ้าตอบ Checkpoint ได้ ถือว่าผ่านวันนั้น
2. ติ๊กช่อง ☐ เมื่อทำเสร็จ แถบสีเหลืองคือวันทบทวน ใช้ตามส่วนที่ค้างได้
3. ทั้งหมดใช้เครื่องมือและแหล่งเรียนรู้ฟรี (รายการอยู่หน้าสุดท้าย)

หมายเหตุทางเทคนิค

- Icarus Verilog รองรับ SystemVerilog ไม่ครบ ให้ใช้ `-g2012` หรือใช้ Verilator ถ้าเจอ Error กับ always_ff/always_comb
- Fork Template ของ Tiny Tapeout ตั้งแต่สัปดาห์ที่ 5 เพื่อไม่ต้องย้ายโค้ดตอนท้าย Template ใช้ cocotb และรัน Flow ผ่าน GitHub Actions
- Flow ขนาดพื้นที่ จำนวนขา และกำหนดส่ง เปลี่ยนตามรอบ ตรวจที่ tinytapeout.com ก่อนเริ่มสัปดาห์ที่ 12
- สัปดาห์ที่ 11 ทางหลักคือดูผล Flow จาก GitHub Actions การติดตั้ง OpenLane บนเครื่องเองเป็นส่วนเสริม

สัปดาห์ 1 · ปูพื้นฐานตัวเลขและวงจรดิจิทัล

สัปดาห์ 2 · Clock, Reset, Flip-Flop และ FSM

สัปดาห์ 3 · เริ่มเขียน Verilog/SystemVerilog

สัปดาห์ 4 · Simulation และการอ่าน Waveform

สัปดาห์ 5 · มินิโปรเจกต์: เขียน Specification

สัปดาห์ 6 · มินิโปรเจกต์: พัฒนา RTL ทีละส่วน

สัปดาห์ 7 · มินิโปรเจกต์: Testbench

สัปดาห์ 8 · มินิโปรเจกต์: Debug และจัดผลงาน

สัปดาห์ 9 · ทดสอบ Corner Cases

สัปดาห์ 10 · ยกระดับ Debug และ Verification

สัปดาห์ 11 · ทดลอง Open-Source ASIC Flow

สัปดาห์ 12 · Tiny Tapeout และ Portfolio

สัปดาห์ที่ 1 · ปูพื้นฐานตัวเลขและวงจรดิจิทัล

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
1	**Binary, Hexadecimal**	ระบบเลขฐาน 2/16 และการแปลงเลขฐาน	แปลงเลข 10 ค่าไปมาระหว่างฐาน 10/2/16	อธิบายได้ว่าทำไม 0xFF = 255	Nandland	☐
2	**Two's Complement**	เลขติดลบในฮาร์ดแวร์ และ Overflow	หาค่า −5 แบบ 4 บิตและ 8 บิต	บอกช่วงค่าของเลข 8 บิตแบบมีเครื่องหมายได้	Nandland	☐
3	**Logic Gates + Truth Table**	AND, OR, NOT, XOR, NAND, NOR	ทำ HDLBits ส่วน Basic Gates	เขียน Truth Table ของ XOR ได้โดยไม่เปิดดู	HDLBits	☐
4	**Boolean Algebra / K-map**	กฎ De Morgan และการลดรูปด้วย K-map	ลดรูปสมการ 3 ข้อ	ลดรูปสมการ 3 ตัวแปรได้	Harris & Harris บทที่ 2	☐
5	**Adder, Mux, Decoder**	Combinational Logic พื้นฐาน	วาด Full Adder และ Mux 2:1	ต่อ Full Adder 4 ตัวเป็น 4-bit Adder ได้	Harris & Harris บทที่ 2	☐
6	**ทบทวนสัปดาห์ที่ 1**	ทวนเนื้อหาวันที่ 1–5	แบบทดสอบสั้น 10 ข้อ	ทำได้ 8/10 ข้อขึ้นไป	—	☐
7	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** มองวงจรแล้วอธิบายได้ว่า Input แต่ละตัวส่งผลต่อ Output อย่างไร

สัปดาห์ที่ 2 · Clock, Reset, Flip-Flop และ FSM

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
8	**Combinational vs Sequential**	วงจรที่ต้อง 'จำสถานะ' ต่างจากวงจรทั่วไปอย่างไร	ยกตัวอย่างวงจรในชีวิตจริงประเภทละ 3 ตัวอย่าง	อธิบายความต่างได้ใน 2 ประโยค	Harris & Harris บทที่ 3	☐
9	**Flip-Flop, Clock, Rising Edge**	D Flip-Flop และ Edge-triggered	วาด Timing Diagram ของ D-FF 8 รอบ Clock	ทำนาย Q ได้ถูกทุกรอบ	Harris & Harris บทที่ 3	☐
10	**Reset: Sync vs Async**	ความต่างของ Reset สองแบบในเชิงแนวคิด	วาด Waveform ของ Reset ทั้งสองแบบ	บอกได้ว่าแบบไหน Reset ทันทีโดยไม่รอ Clock	ChipVerify	☐
11	**Register และ Counter**	Register หลายบิต และ Counter 0–9	ออกแบบ Counter 0–9 แล้ววนกลับ (วาดบนกระดาษ)	อธิบายได้ว่าเงื่อนไขวนกลับเกิดตอนไหน	Harris & Harris บทที่ 3	☐
12	**FSM + Setup/Hold Time**	State, Transition, Output และ Timing เชิงแนวคิด	วาด State Diagram ไฟจราจร	อธิบายได้ว่าจะเกิดอะไรขึ้นถ้า Reset ไม่ทำงาน	Harris & Harris บทที่ 3	☐
13	**ทบทวนสัปดาห์ที่ 2**	ทวน FSM และ Timing	วาด State Diagram ระบบใหม่ 1 ระบบ (เช่น ตู้ขายน้ำ)	วาดได้ครบโดยไม่มี State ค้าง	—	☐
14	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** วาด State Diagram ของระบบเล็ก ๆ ได้ก่อนเขียนโค้ด

สัปดาห์ที่ 3 · เริ่มเขียน Verilog/SystemVerilog

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
15	**Module, Port, wire/logic**	โครงสร้างโมดูลและชนิดสัญญาณ	ทำ HDLBits ส่วน Getting Started + Vectors	เขียนโมดูลที่มี Input/Output ได้เอง	HDLBits	☐
16	**assign และ always_comb**	Continuous Assignment และ Combinational Block	เขียน Mux 2:1 และ 4-bit Adder	บอกได้ว่าโค้ดแต่ละบรรทัดกลายเป็นวงจรอะไร	HDLBits, ChipVerify	☐
17	**always_ff, if, case**	Sequential Block และการกำหนดค่าเริ่มต้นเพื่อกัน Latch	เขียน Counter 0–9	อธิบายได้ว่าทำไมเงื่อนไขไม่ครบจึงเกิด Latch	HDLBits, ChipVerify	☐
18	**Blocking vs Non-blocking**	เครื่องหมาย = กับ <= ต่างกันอย่างไรใน Sequential Logic	เขียน Shift Register ทั้งสองแบบแล้วเทียบผล	อธิบายได้ว่าทำไมใน always_ff ควรใช้ <=	ChipVerify	☐
19	**Parameter + FSM เป็น RTL**	Parameter, Bit Width และการเขียน FSM	แปลง State Diagram ไฟจราจรเป็น RTL	FSM มี default ครบทุก State	HDLBits ส่วน FSM	☐
20	**ทบทวนสัปดาห์ที่ 3**	ทวนกฎการเขียน RTL	ทำโจทย์ HDLBits ที่ยังไม่ผ่านให้ผ่าน	ผ่านโจทย์ FSM ใน HDLBits อย่างน้อย 3 ข้อ	HDLBits	☐
21	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** เขียนโมดูลเล็ก ๆ และอธิบายได้ว่าแต่ละส่วนเป็นวงจรอะไร

สัปดาห์ที่ 4 · Simulation และการอ่าน Waveform

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
22	**ติดตั้งเครื่องมือ**	Icarus Verilog / Verilator และ GTKWave	ติดตั้งและรัน Hello World	รันได้โดยไม่มี Error (Icarus ใช้ -g2012)	Icarus, Verilator, GTKWave	☐
23	**Testbench + Clock Generation**	โครงสร้าง Testbench และการสร้าง Clock	เขียน Testbench สำหรับ Mux	เปิด Waveform (.vcd) ใน GTKWave ได้	ChipVerify	☐
24	**Stimulus และการอ่าน Waveform**	Signal, State, Timestamp	ทดสอบ Counter ให้นับถึงค่าสูงสุดแล้ววนกลับ	ชี้ได้ว่า Counter วนกลับที่ Clock รอบไหน	GTKWave	☐
25	**Reset ระหว่างทำงาน**	พฤติกรรมเมื่อ Reset กลางคัน	สลับ Reset ระหว่างที่ Counter กำลังนับ	อธิบายได้ว่าทำไม Output เปลี่ยนที่รอบนั้น	—	☐
26	**ล่าบั๊กจาก Waveform**	RTL สำหรับสังเคราะห์ vs โค้ดสำหรับทดสอบ	ใส่บั๊ก 3 จุดลงในดีไซน์ แล้วหาจาก Waveform	หาบั๊กเจอครบ 3 จุด	—	☐
27	**ทบทวนสัปดาห์ที่ 4**	ทวน Flow: เขียน → รัน → อ่าน Waveform	ทดสอบ FSM ไฟจราจรจากสัปดาห์ที่ 3	FSM ไฟจราจรทำงานถูกต้องใน Simulation	—	☐
28	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** อธิบายได้ว่าทำไม Output เปลี่ยนที่ Clock รอบนั้น

สัปดาห์ที่ 5 · มินิโปรเจกต์: เขียน Specification

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
29	**เลือกโปรเจกต์**	PWM, UART TX, SPI, FIFO หรือ Timer	เลือก 1 ชิ้น (แนะนำ PWM หรือ UART TX)	เขียนได้ใน 1 ประโยคว่าโมดูลแก้ปัญหาอะไร	—	☐
30	**Fork Template ของ Tiny Tapeout**	โครงสร้างโปรเจกต์ของ TT และข้อจำกัดของรอบปัจจุบัน	Fork Template และอ่าน README	รู้ว่าต้องวาง RTL และ Test ไว้ตรงไหน	Tiny Tapeout	☐
31	**Interface**	Input/Output, Clock และ Reset	เขียนตาราง Port ทั้งหมดพร้อม Bit Width	ทุก Port มีชื่อ ทิศทาง และความกว้าง	—	☐
32	**พฤติกรรมและสถานะ**	State, เงื่อนไขเริ่ม/หยุด และกรณีผิดปกติ	เขียนรายการ State และเงื่อนไข	ไม่มีสถานการณ์ใดที่ไม่รู้ว่าจะไป State ไหน	—	☐
33	**Block + State Diagram**	แยก Datapath และ Control Path	วาด Block Diagram และ State Diagram	ทุกเส้นในไดอะแกรมมีชื่อสัญญาณ	—	☐
34	**ทบทวน Specification**	ตรวจ Spec ให้ครบก่อนเขียนโค้ด	ให้เพื่อนอ่าน Spec แล้วถามคำถาม	ตอบคำถามได้โดยไม่ต้องแก้ Spec ใหม่ทั้งหมด	—	☐
35	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** มี Spec, Block Diagram และ State Diagram ครบ

สัปดาห์ที่ 6 · มินิโปรเจกต์: พัฒนา RTL ทีละส่วน

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
36	**Datapath**	Counter / Register ที่จำเป็น	เขียน Datapath	ทดสอบ Datapath แยกได้	—	☐
37	**Control Path (FSM)**	FSM ตาม State Diagram	เขียน FSM	ทุก State มี default และ Transition ครบ	—	☐
38	**เชื่อมต่อโมดูล**	Top-level และการต่อสัญญาณ	รวม Datapath + FSM	Compile ผ่านโดยไม่มี Warning เรื่อง Width	Verilator --lint-only	☐
39	**ตรวจ Bit Width และ Default**	Lint และ Latch	รัน Lint แล้วแก้ทุก Warning	Lint สะอาด	Verilator --lint-only	☐
40	**ทดสอบทีละฟังก์ชัน**	Smoke Test	Testbench สั้น ๆ ทดสอบกรณีปกติ 1 กรณี	เห็น Output ถูกต้องใน Waveform	GTKWave	☐
41	**ทบทวนสัปดาห์ที่ 6**	ทวนโค้ดเทียบกับ Spec	ทำตาราง 'Spec ข้อนี้ → โค้ดบรรทัดนี้'	ทุกข้อใน Spec มีโค้ดรองรับ	—	☐
42	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** RTL ครบตาม Spec และ Lint สะอาด

สัปดาห์ที่ 7 · มินิโปรเจกต์: Testbench

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
43	**กรณีทำงานปกติ**	Directed Test	เขียน Test กรณีปกติให้ครบ	ผ่านทุกกรณีปกติ	—	☐
44	**Reset ก่อนเริ่มงาน**	ลำดับการ Reset ใน Testbench	เพิ่ม Test การ Reset	ค่าเริ่มต้นถูกต้องหลัง Reset	—	☐
45	**Input เปลี่ยนติดกัน**	Back-to-back Stimulus	เพิ่ม Test ที่ Input เปลี่ยนติดกันหลายรอบ	ไม่มีข้อมูลหาย	—	☐
46	**เริ่ม/หยุดซ้ำ**	Start/Stop หลายรอบ	เพิ่ม Test เริ่มและหยุดซ้ำ	ผ่านทุกรอบ	—	☐
47	**ตรวจผลอัตโนมัติ**	Self-checking Testbench / cocotb	ย้าย Test ไปเป็น cocotb ตาม Template ของ TT	Test รายงาน PASS/FAIL เอง	cocotb, Tiny Tapeout	☐
48	**ทบทวนสัปดาห์ที่ 7**	ทวน Testbench	รัน Test ทั้งหมดด้วยคำสั่งเดียว	ผ่านทั้งหมด	—	☐
49	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** Testbench ตรวจผลอัตโนมัติ และรันได้ด้วยคำสั่งเดียว

สัปดาห์ที่ 8 · มินิโปรเจกต์: Debug และจัดผลงาน

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
50	**แก้บั๊กจาก Waveform**	Debug อย่างเป็นระบบ	แก้บั๊กที่ค้างทั้งหมด	ไม่มี Test ที่ FAIL	GTKWave	☐
51	**Bug Log**	บันทึกปัญหาและวิธีแก้	เขียน Bug Log อย่างน้อย 3 รายการ	ทุกรายการมี อาการ → สาเหตุ → วิธีแก้	—	☐
52	**จัดโฟลเดอร์**	แยก RTL, Testbench และเอกสาร	จัดโครงสร้างโฟลเดอร์ให้เป็นระเบียบ	คนอื่นหาไฟล์ได้เองโดยไม่ต้องถาม	—	☐
53	**README**	วิธีรันซ้ำและผลที่คาดหวัง	เขียน README	เพื่อนรันตาม README ได้สำเร็จ	—	☐
54	**Waveform + สรุปผล**	ภาพประกอบและสรุปผลการทดสอบ	แนบภาพ Waveform และตารางผล Test	มี Spec, RTL, Test, Waveform และคำอธิบายครบ	—	☐
55	**Checkpoint ครึ่งทาง**	ทวนสัปดาห์ที่ 5–8	Push ขึ้น GitHub	GitHub Actions ของ Template รันผ่าน	Tiny Tapeout	☐
56	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** มีโปรเจกต์ที่มี Spec, RTL, Testbench, Waveform และ README ครบ

สัปดาห์ที่ 9 · ทดสอบ Corner Cases

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
57	**Reset ระหว่างทำงาน**	Reset กลางคันในทุก State	เพิ่ม Test Reset ใน State ต่าง ๆ	กลับสู่ค่าเริ่มต้นทุกครั้ง	—	☐
58	**Input ติดกันไม่มีช่วงพัก**	Back-to-back ที่ความเร็วสูงสุด	เพิ่ม Test แบบไม่มีช่วงพัก	ไม่มีข้อมูลหายหรือซ้ำ	—	☐
59	**ค่าต่ำสุด/สูงสุด, เต็ม/ว่าง**	Boundary Condition	Test Counter ที่ขอบ หรือ FIFO เต็ม/ว่าง	พฤติกรรมตรง Spec ที่ขอบ	—	☐
60	**คำสั่งใหม่ก่อนงานเดิมเสร็จ**	การจัดการคำสั่งซ้อน	เพิ่ม Test คำสั่งซ้อน	ผลตรงกับที่ Spec กำหนด	—	☐
61	**Enable ใกล้ขอบ / ค่าที่ไม่ใช้**	Enable timing และ Input ผิดปกติ	เพิ่ม Test ทั้งสองแบบ	ไม่เข้า State ค้าง	—	☐
62	**ตาราง Test Plan**	เชื่อม Test กลับไปที่ Spec	ทำตาราง Test Case → ข้อใน Spec	ทุก Test รู้ว่ากำลังตรวจอะไร	—	☐
63	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** มี Test Plan ที่เชื่อมกลับไปยัง Spec

สัปดาห์ที่ 10 · ยกระดับ Debug และ Verification

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
64	**Expected Output**	Reference Model อย่างง่าย	เขียน Model ใน Python เทียบผลใน cocotb	ทุก Test เทียบกับ Expected อัตโนมัติ	cocotb	☐
65	**Assertion**	ตรวจ State หรือเงื่อนไขที่ห้ามเกิด	เพิ่ม Assertion 2–3 จุด	Assertion จับบั๊กที่ตั้งใจใส่ได้	ChipVerify	☐
66	**แยก Test Case**	ตั้งชื่อและแยก Test ให้อ่านง่าย	Refactor Test เป็นฟังก์ชันย่อย	ระบุได้ทันทีว่า Test ไหน FAIL	cocotb	☐
67	**Test Summary + Script**	รันซ้ำด้วยคำสั่งเดียว	ทำ Makefile/Script สรุป PASS/FAIL	ผลการรันซ้ำเหมือนเดิมทุกครั้ง	cocotb	☐
68	**Functional Coverage (แนวคิด)**	เราทดสอบพฤติกรรมอะไรไปแล้วบ้าง	ทำตารางว่าพฤติกรรมไหนยังไม่ถูกทดสอบ	รู้ว่ามีช่องว่างตรงไหน	ChipVerify	☐
69	**ทบทวนสัปดาห์ที่ 10**	Test ผ่านเพราะดีไซน์ถูก หรือ Test ยังไม่ครอบคลุม?	ใส่บั๊กใหม่ 2 จุดแล้วดูว่า Test จับได้ไหม	Test จับบั๊กได้ทั้ง 2 จุด	—	☐
70	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** การทดสอบมีผลชัดเจนและทำซ้ำได้

สัปดาห์ที่ 11 · ทดลอง Open-Source ASIC Flow

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
71	**ภาพรวม RTL-to-GDSII**	Synthesis → Floorplan → Place → CTS → Route → STA → DRC/LVS → GDSII	วาด Flow เป็นแผนภาพของตัวเอง	อธิบายแต่ละขั้นได้ใน 1 ประโยค	Tiny Tapeout, OpenLane docs	☐
72	**Synthesis ด้วย Yosys**	RTL → Gate-level Netlist	รัน Yosys กับดีไซน์ของตัวเอง แล้วดูจำนวน Cell	อธิบายได้ว่า Cell ส่วนใหญ่มาจากส่วนไหนของโค้ด	Yosys	☐
73	**อ่านผล Flow จาก GitHub Actions**	Log, Warning และ Error	เปิด Log ของ GitHub Actions ใน Repo ของตัวเอง	สรุปได้ว่ามี Warning อะไรบ้าง	Tiny Tapeout	☐
74	**Timing Report**	Slack, Critical Path	อ่าน Timing Report จากผล Flow	บอกได้ว่า Slack บวกหรือลบ และ Path ไหนวิกฤต	Tiny Tapeout	☐
75	**ดู Layout ใน KLayout**	GDS, Layer และ Standard Cell	เปิดไฟล์ GDS ใน KLayout	ชี้ได้ว่าดีไซน์ของตัวเองอยู่ตรงไหน	KLayout	☐
76	**ทบทวน / (เสริม) ติดตั้ง OpenLane เอง**	สำหรับคนที่มีเวลาเพิ่ม	(ไม่บังคับ) รัน OpenLane บนเครื่องตัวเอง	เห็นว่า RTL, Netlist และ Layout คือดีไซน์เดียวกัน	OpenLane docs	☐
77	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** เห็นว่า RTL, Netlist และ Layout คือดีไซน์เดียวกัน

สัปดาห์ที่ 12 · Tiny Tapeout และ Portfolio

วันที่	หัวข้อ	📖 เรียน	🛠️ ลงมือ	✅ Checkpoint	แหล่งเรียนรู้	☐
78	**ตรวจข้อกำหนดของรอบที่จะสมัคร**	ขนาด, จำนวนขา และ Interface ของรอบปัจจุบัน	เทียบดีไซน์กับข้อกำหนดของ TT	ไม่มีข้อใดเกินข้อจำกัด	Tiny Tapeout	☐
79	**Checklist ทางเทคนิค**	Spec, RTL, Test, Warning จาก Flow	ไล่ Checklist ทีละข้อ	ทุกข้อติ๊กผ่าน	—	☐
80	**Portfolio: ปัญหาและสถาปัตยกรรม**	ปัญหาที่แก้ และเหตุผลการออกแบบ	เขียนส่วนนี้พร้อม Block/State Diagram	คนนอกสายอ่านแล้วเข้าใจว่าทำอะไร	—	☐
81	**Portfolio: การทดสอบและบทเรียน**	วิธีทดสอบ ผลลัพธ์ และบั๊กที่เจอ	เขียนส่วนนี้จาก Bug Log และ Test Summary	มีบั๊กจริงอย่างน้อย 1 เรื่อง พร้อมวิธีหาสาเหตุ	—	☐
82	**แผนเวอร์ชันถัดไป**	สิ่งที่จะปรับปรุง	เขียนรายการ Next Steps	มีอย่างน้อย 3 ข้อที่ทำได้จริง	—	☐
83	**Checkpoint สุดท้าย**	ทวนทั้ง 12 สัปดาห์	เผยแพร่ Repo และ Portfolio	พร้อมส่ง Tiny Tapeout รอบถัดไป	Tiny Tapeout	☐
84	พัก					

🎯 **เป้าหมายปลายสัปดาห์:** พร้อมส่ง Tiny Tapeout และมี Portfolio

แหล่งเรียนรู้

ชื่อ	สัปดาห์	ใช้ทำอะไร	ลิงก์
HDLBits	1–3	ฝึก Verilog ในเบราว์เซอร์ ไม่ต้องติดตั้ง	hdlbits.01xz.net
Nandland	1–4	บทเรียนพื้นฐานดิจิทัลและ HDL สำหรับมือใหม่	nandland.com
ChipVerify	2–4, 10	อ้างอิง Verilog/SystemVerilog และ Assertion	chipverify.com
Harris & Harris	1–3	Digital Design and Computer Architecture (บทที่ 2–4)	—
Icarus Verilog	4+	Simulator (ใช้ -g2012; รองรับ SystemVerilog ไม่ครบ)	steveicarus.github.io/iverilog
Verilator	4+	Simulator และ Lint (รองรับ SystemVerilog ได้ดีกว่า)	veripool.org/verilator
GTKWave	4+	ดู Waveform (.vcd)	gtkwave.sourceforge.net
cocotb	7, 10	Testbench ด้วย Python (Template ของ Tiny Tapeout ใช้)	cocotb.org
Tiny Tapeout	5, 8, 11–12	Template, ข้อกำหนดแต่ละรอบ และ Flow ผ่าน GitHub Actions	tinytapeout.com
Yosys	11	Synthesis	yosyshq.net/yosys
OpenLane docs	11	RTL-to-GDSII Flow (ส่วนเสริม)	openlane2.readthedocs.io
KLayout	11	ดูไฟล์ Layout (GDS)	klayout.de

Virtuchip.AI Academy · เรียนรู้ • ปรับตัว • ก้าวไปข้างหน้า